



OBJECT-ORIENTED PROGRAMMING (OOP)



Task 1: What is Object-Oriented Programming (OOP)?

Object-Oriented Programming (OOP) is a programming style that organizes a program around **objects**. An object combines **data** (attributes) and **behavior** (methods) in one unit. Objects are created from **classes**.

Four main pillars:

Pillar	Meaning
Encapsulation	Keep data and the methods that use it together, and protect the data
Abstraction	Show only what is necessary and hide the details
Inheritance	A class reuses the features of another class
Polymorphism	The same method name can behave differently for different objects

Benefits: code reuse, easier maintenance, better organization, and closer to real-world thinking.

```
class Dog:
    def __init__(self, name):
        self.name = name          # data (attribute)

    def bark(self):               # behavior (method)
        print(f"{self.name} says Woof!")

my_dog = Dog("Rex")              # object
my_dog.bark()
```

Task 2: Difference between Class and Object

Feature	Class	Object
Definition	A blueprint/template	A real instance created from the class
Memory	No memory for data until an object is created	Uses memory for its own data
Creation	Written once with <code>class</code>	Created many times with <code>ClassName()</code>
Example	<code>Car</code>	<code>my_car, your_car</code>

```
class Car:                        # class
    def __init__(self, color):
        self.color = color

car1 = Car("red")                 # object 1
car2 = Car("blue")               # object 2
```

Task 3: Difference between Encapsulation and Abstraction

Feature	Encapsulation	Abstraction
Idea	Wrapping data and methods together and protecting the data	Hiding complexity and showing only essential features
Focus	How data is protected	What an object does
Achieved by	Private/protected attributes, getters/setters	Abstract classes and abstract methods
Example	Bank balance is private	You use <code>withdraw()</code> without knowing how it works inside

```
from abc import ABC, abstractmethod

class Account:                                # Encapsulation
    def __init__(self):
        self.__balance = 0                    # hidden data
    def deposit(self, amount):
        if amount > 0:
            self.__balance += amount

class Shape(ABC):                              # Abstraction
    @abstractmethod
    def area(self):
        pass
```

Task 4: Difference between Inheritance and Composition

Feature	Inheritance	Composition
Relationship	"is-a"	"has-a"
Meaning	A class extends another class	A class contains objects of other classes
Coupling	Tighter	Looser
Example	Dog is an Animal	Car has an Engine

```
class Animal: pass
class Dog(Animal): pass                # Inheritance (Dog is an Animal)

class Engine: pass
class Car:
    def __init__(self):
        self.engine = Engine()        # Composition (Car has an Engine)
```

Task 5: Difference between Method Overloading and Method Overriding

Feature	Method Overloading	Method Overriding
Meaning	Same method name, different parameters , in the same class	Child class redefines a parent method with the same name and parameters
Needs inheritance?	No	Yes
Python support	Not directly (last definition wins). Use default arguments or <code>*args</code>	Fully supported
Binding	Compile-time idea	Run-time

```
# Overloading (simulated with default arguments)
class Calc:
    def add(self, a, b=0, c=0):
        return a + b + c

# Overriding
class Animal:
    def speak(self):
        return "Some sound"

class Cat(Animal):
    def speak(self):          # overrides the parent method
        return "Meow"
```

Task 6: Difference between Compile-Time and Run-Time Polymorphism

Feature	Compile-Time Polymorphism	Run-Time Polymorphism
Also called	Static polymorphism	Dynamic polymorphism
Decided	When the code is compiled	While the program runs
Achieved by	Method overloading, operator overloading	Method overriding
Speed	Slightly faster	Slightly slower
Python	Only simulated (Python is interpreted)	Natural in Python

```
class Animal:
    def speak(self): print("...")
class Dog(Animal):
    def speak(self): print("Woof")
class Cat(Animal):
    def speak(self): print("Meow")

for animal in [Dog(), Cat()]:
    animal.speak()          # method chosen at run time
```

Task 7: Difference between Abstract Class and Interface

Feature	Abstract Class	Interface
Methods	Abstract and normal methods	Only method declarations (in classic definition)
Attributes	Can have attributes	Usually none
Constructor	Can have one	No
Inheritance	A class usually inherits one abstract class	A class can implement many interfaces
Purpose	Share common code + force some methods	Define a contract only

Python note: Python has no `interface` keyword. We use `ABC` with only abstract methods to create an interface-like class.

```
from abc import ABC, abstractmethod

class Vehicle(ABC):
    def start(self):
        print("Starting...")
    @abstractmethod
    def wheels(self): pass

class Payable(ABC):
    @abstractmethod
    def pay(self): pass
```

Task 8: Difference between Association, Aggregation, and Composition (Conceptual only)

Feature	Association	Aggregation	Composition
Meaning	Objects are simply related/use each other	"Has-a" — whole and parts, parts can live alone	"Has-a" — whole owns parts, parts die with the whole
Strength	Weakest	Medium	Strongest
Lifetime	Independent	Independent	Dependent on the owner
Example	Teacher ↔ Student	Department has Teachers	House has Rooms

- **Association:** a teacher teaches students.
- **Aggregation:** if the department is deleted, the teachers still exist.
- **Composition:** if the house is destroyed, its rooms are gone too.

Task 9: Difference between Static Members and Instance Members

Feature	Static (Class) Members	Instance Members
Belongs to	The class	Each object
Copies	One shared copy	One copy per object
Access	ClassName.member	object.member
Uses self?	No (@staticmethod) or cls (@classmethod)	Yes

```
class Student:
    count = 0                # static (class) attribute

    def __init__(self, name):
        self.name = name    # instance attribute
        Student.count += 1

    @staticmethod
    def school_name():       # static method
        return "ABC University"

s1 = Student("Ali"); s2 = Student("Sara")
print(Student.count)        # 2 (shared)
print(s1.name, s2.name)    # different for each object
```

Task 10: Difference between Constructor and Destructor

Feature	Constructor	Destructor
Purpose	Initialize a new object	Clean up before an object is destroyed
Called	When the object is created	When the object is about to be destroyed
Python method	<code>__init__</code>	<code>__del__</code>
Parameters	Can take parameters	Only <code>self</code>
Common use	Set attribute values	Close files/connections

```
class Demo:
    def __init__(self):
        print("Object created")
    def __del__(self):
        print("Object destroyed")

d = Demo()
del d
```

Task 11: Difference between Access Modifiers

Access modifiers control **who can access** the attributes and methods of a class.

Modifier	Python Syntax	Access Level	Meaning
Public	name	Everywhere	Anyone can use it
Protected	<code>_name</code>	Class and subclasses (by convention)	Should not be used outside
Private	<code>__name</code>	Inside the class only (name mangling)	Hidden from outside

```
class Person:
    def __init__(self):
        self.name = "Ali"      # public
        self._age = 20        # protected (convention)
        self.__id = 123       # private

p = Person()
print(p.name)                # works
# print(p.__id)              # AttributeError
```

Note: Python does not enforce strict access; protected is only a convention, and private uses name mangling.

Task 12: Difference between Mutable and Immutable Objects

Feature	Mutable	Immutable
Can change after creation?	Yes	No
Memory	Same object is modified	A new object is created on change
Examples in Python	<code>list, dict, set</code>	<code>int, float, str, tuple, bool</code>

```
numbers = [1, 2, 3]
numbers.append(4)      # OK, same list changed

text = "hello"
# text[0] = "H"        # TypeError
text = "H" + text[1:]  # creates a NEW string
```

Task 13: Difference between Shallow Copy and Deep Copy

Feature	Shallow Copy	Deep Copy
Copies	The outer object only	The outer object and all nested objects
Nested objects	Shared with the original	Independent
Function	<code>copy.copy()</code>	<code>copy.deepcopy()</code>
Speed	Faster	Slower

```
import copy

original = [[1, 2], [3, 4]]
shallow = copy.copy(original)
deep = copy.deepcopy(original)

original[0][0] = 99
print(shallow[0][0])    # 99 (shared inner list)
print(deep[0][0])       # 1  (independent)
```

Task 14: Difference between Tight Coupling and Loose Coupling

Feature	Tight Coupling	Loose Coupling
Dependency	Classes depend strongly on each other	Classes depend on each other very little
Changing one class	Forces changes in others	Others are usually not affected
Reusability / testing	Hard	Easy
Quality	Poor design	Good design

```
# Tight: Car creates a specific Engine itself
class Car:
    def __init__(self):
        self.engine = PetrolEngine()

# Loose: Car accepts any engine
class Car:
    def __init__(self, engine):
        self.engine = engine
```

Task 15: Difference between Dependency Injection and Direct Dependency (Basic concept only)

Feature	Direct Dependency	Dependency Injection
Who creates the dependency?	The class itself	It is given from outside
Coupling	Tight	Loose
Testing	Harder	Easier (you can pass a fake object)

```
class Printer:
    def print_text(self, text): print(text)

# Direct dependency
class Report:
    def __init__(self):
        self.printer = Printer()          # created inside

# Dependency injection
class Report:
    def __init__(self, printer):
        self.printer = printer           # given from outside

report = Report(Printer())
```

Task 16: Difference between OOP and Procedural Programming

Feature	OOP	Procedural Programming
Organized around	Objects and classes	Functions/procedures
Data and functions	Combined together	Separate
Data security	Better (encapsulation)	Data is often global and open
Reuse	Inheritance	Calling functions
Best for	Large and complex programs	Small, simple programs
Examples	Python (OOP style), Java	C, basic Python scripts

```
# Procedural
def area(width, height):
    return width * height

# OOP
class Rectangle:
    def __init__(self, width, height):
        self.width, self.height = width, height
    def area(self):
        return self.width * self.height
```

Task 17: Difference between OOP and Functional Programming

Feature	OOP	Functional Programming
Main building block	Objects	Functions
State	Objects keep and change state	Avoids changing state
Data	Mutable (usually)	Immutable (preferred)
Side effects	Common	Avoided (pure functions)
Reuse	Inheritance / composition	Function composition, higher-order functions
Examples	Java, C++, Python	Haskell, Lisp, Python (map, filter, lambda)

```
# Functional style
numbers = [1, 2, 3, 4]
squares = list(map(lambda x: x * x, numbers))
```

Task 18: Difference between Runtime Binding and Compile-Time Binding

Binding means connecting a method call to the method's code.

Feature	Compile-Time (Early/Static) Binding	Runtime (Late/Dynamic) Binding
When	During compilation	While the program runs
Based on	Reference type	Actual object type
Speed	Faster	Slightly slower
Related to	Overloading	Overriding
Python	Not typical	Always used

```
class Animal:
    def speak(self): print("...")
class Dog(Animal):
    def speak(self): print("Woof")

a = Dog()
a.speak()      # Python decides at run time -> "Woof"
```

Task 19: Difference between Multiple Inheritance and Multilevel Inheritance

Feature	Multiple Inheritance	Multilevel Inheritance
Structure	One child has more than one parent	A chain: grandparent → parent → child
Diagram	A, B → C	A → B → C
Python support	Yes	Yes
Risk	Diamond problem / confusion (Python uses MRO)	Deep chains can be hard to follow

```
# Multiple inheritance
class Father: pass
class Mother: pass
class Child(Father, Mother): pass

# Multilevel inheritance
class Animal: pass
class Mammal(Animal): pass
class Dog(Mammal): pass
```

Task 20: Difference between Garbage Collection and Manual Memory Management

Feature	Garbage Collection	Manual Memory Management
Who frees memory?	The runtime, automatically	The programmer
Ease of use	Easy	Harder
Common errors	Rare	Memory leaks, dangling pointers
Control	Less	Full
Performance	Small overhead from the collector	Can be faster if done well
Languages	Python, Java, C#	C, C++ (malloc/free, new/delete)

Python uses **reference counting** plus a cycle-detecting garbage collector.

```
import gc

x = [1, 2, 3]
x = None          # no reference left -> memory is freed automatically
gc.collect()      # optional: force a collection
```

References

- [1] Python Software Foundation. (n.d.). *Classes* (The Python Tutorial). Python 3 Documentation. <https://docs.python.org/3/tutorial/classes.html> (Used for: Tasks 1–5, 9, 11, 19)
- [2] Python Software Foundation. (n.d.). *abc — Abstract Base Classes*. Python 3 Documentation. <https://docs.python.org/3/library/abc.html> (Used for: Tasks 3, 7)
- [3] Python Software Foundation. (n.d.). *Data model* (The Python Language Reference). Python 3 Documentation. <https://docs.python.org/3/reference/datamodel.html> (Used for: Tasks 5, 6, 10, 12, 18)
- [4] Python Software Foundation. (n.d.). *copy — Shallow and deep copy operations*. Python 3 Documentation. <https://docs.python.org/3/library/copy.html> (Used for: Task 13)
- [5] Python Software Foundation. (n.d.). *gc — Garbage Collector interface*. Python 3 Documentation. <https://docs.python.org/3/library/gc.html> (Used for: Task 20)
- [6] Python Software Foundation. (n.d.). *Functional Programming HOWTO*. Python 3 Documentation. <https://docs.python.org/3/howto/functional.html> (Used for: Task 17)
- [7] Python Software Foundation. (n.d.). *The Python 2.3 Method Resolution Order*. Python 3 Documentation. <https://docs.python.org/3/howto/mro.html> (Used for: Task 19)
- [8] Oracle. (n.d.). *Lesson: Object-Oriented Programming Concepts*. The Java Tutorials. <https://docs.oracle.com/javase/tutorial/java/concepts/index.html> (Used for: Tasks 1–3, 8)
- [9] Oracle. (n.d.). *Creating and Using Interfaces*. The Java Tutorials. <https://docs.oracle.com/javase/tutorial/java/landl/createinterface.html> (Used for: Task 7)
- [10] Fowler, M. (2004, January 23). *Inversion of Control Containers and the Dependency Injection pattern*. <https://martinfowler.com/articles/injection.html> (Used for: Tasks 14, 15)
- [11] Real Python. (n.d.). *Object-Oriented Programming (OOP) in Python*. <https://realpython.com/python3-object-oriented-programming/> (Used for: Tasks 1, 2, 9, 11)
- [12] Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley. (Used for: Tasks 4, 14)
- [13] Booch, G., Maksimchuk, R. A., Engle, M. W., Young, B. J., Conallen, J., & Houston, K. A. (2007). *Object-Oriented Analysis and Design with Applications* (3rd ed.). Addison-Wesley. (Used for: Tasks 1, 8)